Universidade Federal de Mato Grosso

Disciplina: Estrutura de Dados 1 Docente: Dr. Ivairton

Discente: Humberto Henrique Castro Rocha Matrícula: 202512722003

Discente: Paulo Henrique Gonçalves Coelho Matrícula: 202511722024

Relatório — *22 de dezembro de 2025*

Análise dos métodos de ordenação

Metodologia

Geração dos Dados (Etapa 1)

Conforme especificado, foram gerados arquivos contendo números inteiros positivos pseudoaleatórios entre 1 e 999.999. Para garantir a unicidade dos valores (sem repetições), foi utilizado o algoritmo *Fisher-Yates Shuffle*.

A lógica implementada consiste em criar um vetor ordenado contendo todos os números possíveis e, em seguida, embaralhar este vetor. Foram gerados 5 arquivos diferentes para cada tamanho de entrada (N), totalizando 15 arquivos:

- 5 arquivos com $N = 1.000$
- 5 arquivos com $N = 10.000$
- 5 arquivos com $N = 100.000$

Algoritmos Implementados (Etapa 2)

Os seguintes métodos foram implementados em linguagem C:

1. **Bubble Sort:** Algoritmo simples de complexidade $O(n^2)$, que percorre o vetor repetidamente trocando elementos adjacentes fora de ordem.

2. **Insertion Sort:** Também $O(n^2)$ no pior caso, constrói a ordenação final um item de cada vez.

3. **Shell Sort:** Uma generalização do Insertion Sort que permite a troca de itens distantes, utilizando uma sequência de lacunas (*gaps*) reduzidas pela metade a cada iteração.

4. **Quick Sort:** Algoritmo de divisão e conquista com complexidade média $O(n \log n)$. Utiliza um pivô para particionar o vetor.

5. **Merge Sort:** Algoritmo de divisão e conquista estável, com complexidade garantida de $O(n \log n)$, que divide o vetor e combina as metades ordenadas.

Resultados Obtidos

Abaixo são apresentados os resultados das medições, detalhando as execuções para cada arquivo e a média final por tamanho de entrada.

Tabela 1: Resultados: Bubble Sort

Entrada	**Num. Trocas**	**Tempo (s)**
1000-arq1.txt	244.355	0.0030
1000-arq2.txt	236.642	0.0020
1000-arq3.txt	250.069	0.0010
1000-arq4.txt	248.070	0.0020
1000-arq5.txt	238.969	0.0010
MÉDIA (1.000)	**243.621**	**0.0018**
10000-arq1.txt	22.768.104	0.1050
10000-arq2.txt	23.236.708	0.1020
10000-arq3.txt	23.134.015	0.1090
10000-arq4.txt	23.687.648	0.1070
10000-arq5.txt	23.945.466	0.1070
MÉDIA (10.000)	**23.354.388**	**0.1060**
100000-arq1.txt	1.137.729.494	12.6460
100000-arq2.txt	1.558.784.898	14.8110
100000-arq3.txt	1.828.786.028	16.1460
100000-arq4.txt	2.004.338.662	16.5900
100000-arq5.txt	2.124.369.545	16.6670
MÉDIA (100.000)	**1.730.801.725**	**15.3720**

Tabela 2: Resultados: Insertion Sort

Entrada	**Num. Trocas**	**Tempo (s)**
1000-arq1.txt	244.355	0.0000
1000-arq2.txt	236.642	0.0000
1000-arq3.txt	250.069	0.0010
1000-arq4.txt	248.070	0.0000
1000-arq5.txt	238.969	0.0010
MÉDIA (1.000)	**243.621**	**0.0004**
10000-arq1.txt	22.768.104	0.0310
10000-arq2.txt	23.236.708	0.0270
10000-arq3.txt	23.134.015	0.0280
10000-arq4.txt	23.687.648	0.0280
10000-arq5.txt	23.945.466	0.0300
MÉDIA (10.000)	**23.354.388**	**0.0288**
100000-arq1.txt	1.137.729.494	1.3050
100000-arq2.txt	1.558.784.898	1.7850
100000-arq3.txt	1.828.786.028	2.1100
100000-arq4.txt	2.004.338.662	2.3090
100000-arq5.txt	2.124.369.545	2.4440
MÉDIA (100.000)	**1.730.801.725**	**1.9906**

Tabela 3: Resultados: Shell Sort

Entrada	**Num. Trocas**	**Tempo (s)**
1000-arq1.txt	7.199	0.0010
1000-arq2.txt	7.588	0.0000
1000-arq3.txt	7.795	0.0000
1000-arq4.txt	7.158	0.0010
1000-arq5.txt	7.467	0.0000
MÉDIA (1.000)	**7.441**	**0.0004**
10000-arq1.txt	151.612	0.0020
10000-arq2.txt	156.802	0.0010
10000-arq3.txt	146.171	0.0010
10000-arq4.txt	144.338	0.0010
10000-arq5.txt	153.602	0.0010
MÉDIA (10.000)	**150.505**	**0.0012**
100000-arq1.txt	2.414.086	0.0150
100000-arq2.txt	2.775.824	0.0180
100000-arq3.txt	2.791.240	0.0180
100000-arq4.txt	2.795.134	0.0170
100000-arq5.txt	2.708.555	0.0170
MÉDIA (100.000)	**2.696.968**	**0.0170**

Tabela 4: Resultados: Quick Sort

Entrada	**Num. Trocas**	**Tempo (s)**
1000-arq1.txt	6.275	0.0000
1000-arq2.txt	6.068	0.0000
1000-arq3.txt	6.145	0.0000
1000-arq4.txt	5.971	0.0000
1000-arq5.txt	6.028	0.0000
MÉDIA (1.000)	**6.097**	**0.0000**
10000-arq1.txt	83.540	0.0000
10000-arq2.txt	76.652	0.0010
10000-arq3.txt	83.370	0.0010
10000-arq4.txt	84.061	0.0010
10000-arq5.txt	90.027	0.0000
MÉDIA (10.000)	**83.530**	**0.0006**
100000-arq1.txt	1.132.877	0.0110
100000-arq2.txt	1.145.094	0.0090
100000-arq3.txt	1.185.160	0.0090
100000-arq4.txt	1.355.878	0.0090
100000-arq5.txt	1.154.870	0.0100
MÉDIA (100.000)	**1.194.776**	**0.0096**

Tabela 5: Resultados: Merge Sort

Entrada	**Num. Trocas**	**Tempo (s)**
1000-arq1.txt	9.976	0.0000
1000-arq2.txt	9.976	0.0010
1000-arq3.txt	9.976	0.0010
1000-arq4.txt	9.976	0.0000
1000-arq5.txt	9.976	0.0000
MÉDIA (1.000)	**9.976**	**0.0004**
10000-arq1.txt	133.616	0.0020
10000-arq2.txt	133.616	0.0020
10000-arq3.txt	133.616	0.0020
10000-arq4.txt	133.616	0.0010
10000-arq5.txt	133.616	0.0020
MÉDIA (10.000)	**133.616**	**0.0018**
100000-arq1.txt	1.668.928	0.0180
100000-arq2.txt	1.668.928	0.0180
100000-arq3.txt	1.668.928	0.0200
100000-arq4.txt	1.668.928	0.0200
100000-arq5.txt	1.668.928	0.0210
MÉDIA (100.000)	**1.668.928**	**0.0194**

Análise e Discussão

Comparação de Desempenho

Os resultados obtidos evidenciam de forma clara a distinção entre as classes de complexidade dos algoritmos testados.

Algoritmos Quadráticos ($O(n^2)$): O **Bubble Sort** apresentou o pior desempenho em todos os cenários. No caso de 100.000 registros, sua média de tempo atingiu cerca de 15,37 segundos, com mais de 1,7 bilhão de trocas. O **Insertion Sort**, embora pertença à mesma classe de complexidade, mostrou-se significativamente mais rápido que o Bubble Sort, com uma média de 1,99 segundos para 100.000 registros, confirmando que, na prática, suas otimizações (menor número de escritas e interrupção antecipada do loop interno) trazem grandes benefícios.

Algoritmos Eficientes ($O(n \log n)$ e Otimizados): O salto de desempenho para os algoritmos eficientes é brutal.

- O **Quick Sort** foi o algoritmo mais rápido, ordenando 100.000 registros em impressionantes 0,0096 segundos em média. Isso representa uma velocidade mais de 1.500 vezes superior ao Bubble Sort.
- O **Shell Sort** surpreendeu positivamente, com média de 0,0170 segundos, ficando muito próximo dos algoritmos de divisão e conquista para este tamanho de entrada.
- O **Merge Sort** manteve uma consistência notável (estável em torno de 0,0194 segundos), ligeiramente mais lento que o Quick Sort devido ao *overhead* de alocação de memória auxiliar e cópia de dados.

Taxa de Comparação e Troca

A análise das trocas corrobora a teoria:

- No **Bubble Sort**, o número de trocas cresceu de 243 mil ($N = 1.000$) para 1,7 bilhão ($N = 100.000$), um crescimento quadrático clássico.
- Já no **Quick Sort**, as trocas para $N = 100.000$ foram apenas cerca de 1,19 milhão, demonstrando a eficiência logarítmica na movimentação dos dados.

Conclusão

O experimento valida a importância da escolha adequada do algoritmo de ordenação para grandes volumes de dados. Enquanto métodos simples como Bubble Sort tornam-se inviáveis para $N = 100.000$ (demorando mais de 15 segundos), métodos eficientes como Quick Sort resolvem o mesmo problema quase instantaneamente (menos de 0,01 segundo). O Shell Sort mostrou-se uma alternativa robusta e simples, com desempenho competitivo frente aos algoritmos recursivos mais complexos.

Arquivo: trab.c

Listing 1: Programa completo

```
1
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <time.h> // mede o tempo
5  #include <string.h>
6
7  // Estrutura que armazena as medicoes de tempo e troca
8  typedef struct {
9      long long trocas;
10     double tempo;
11 } Resultado;
12
13 // ---- GERAR OS ARQUIVOS ----
14 // Fiz uma pesquisa na internet e descobri um algoritmo que "sorteia"
       numeros sem repeticao. se chama fisher yates
15 // Bascicamente ele pega um vetor já criado e reorganiza, ou seja, isso
       evita que numeros ja criados sejam novamente criados,
16 // pois ele reorganiza o proprio vetor, sem criar nenhum auxiliar
17
18 void gerarArquivos() {
19     int tamanhos[] = {1000, 10000, 100000};  //eu tentei usar #define
          mas por algum motivo tava dando erro, entao preferi ir assim msm
20     int *pool = malloc(999999 * sizeof(int)); //criamos um ponteiro
21     for (int i = 0; i < 999999; i++){
22         pool[i] = i + 1; // criamos um vetor inicial que vai de 0 até
              999999, e prenchemos em ordem crescente, de qualquer modo
23                          // o algoritmo fisher yates vai embaralhar tudo
                               . Isso só garante que nao tenhamos repetidos
24     }
25
26     srand(time(NULL));
27
28     for (int i = 0; i < 3; i++) {
29         int n = tamanhos[i];
```

```
30          for (int f_index = 1; f_index <= 5; f_index++) {
31              // aqui que entra o fisher yates
32              for (int j = 0; j < n; j++) { // o "n" que vai decidir
                    quantas trocas fazemos, isso garante que o arquivo de mil
                     dados tenha realmente mil dados
33                  int k = j + rand() % (999999 - j);      // sorteia uma
                        posicao pra efetuar a troca, poderiamos fazer essa
                        troca
34                  int temp = pool[j];                     // numa funcao
                        exterior, como no quick sort, usando ponteiros,
35                  pool[j] = pool[k];                      // mas isso
                        adicionaria uma complexidade desnecessaria
36                  pool[k] = temp;
37              }
38
39              // cria o arquivo
40              char nome[30];
41              sprintf(nome, "%d-arq%d.txt", n, f_index);
42              FILE *f = fopen(nome, "w");
43              for (int z = 0; z < n; z++) fprintf(f, "%d\n", pool[z]);
44              fclose(f);
45          }
46      }
47      free(pool);
48  }
49
50  // --- OS ALGORITMOS DE ORDENACAO ---
51
52  Resultado bubbleSort(int *vetor, int tam) { // BUBBLE SORT //
53      long long trocas = 0;
54      clock_t inicio = clock();  // inicia o tempo, primeiro ele marca o
            inicio, para depois subtrair do final
55      for (int i = 0; i < tam - 1; i++) {
56          for (int j = 0; j < tam - i - 1; j++) {
57              if (vetor[j] > vetor[j + 1]) {
58                  int tmp = vetor[j];
59                  vetor[j] = vetor[j + 1];
```

```
60                  vetor[j + 1] = tmp;
61                  trocas++;
62              }
63          }
64      }
65      return (Resultado){trocas, (double)(clock() - inicio) /
            CLOCKS_PER_SEC};
66  }
67
68  Resultado insertionSort(int *vetor, int tam) { // INSERTION SORT //
69      long long trocas = 0;
70      clock_t inicio = clock();
71      for (int i = 1; i < tam; i++) {
72          int chave = vetor[i], j = i - 1;
73          while (j >= 0 && vetor[j] > chave) {
74              vetor[j + 1] = vetor[j];
75              j--;
76              trocas++;
77          }
78          vetor[j + 1] = chave;
79      }
80      return (Resultado){trocas, (double)(clock() - inicio) /
            CLOCKS_PER_SEC};
81  }
82
83  Resultado shellSort(int *vetor, int tam) {
84      long long trocas = 0;
85      clock_t inicio = clock();
86      for (int lacuna = tam / 2; lacuna > 0; lacuna = lacuna / 2) {
87          for (int i = lacuna; i < tam; i++) {
88              int temp = vetor[i], j;
89              for (j = i; j >= lacuna && vetor[j - lacuna] > temp; j -=
                    lacuna) {
90                  vetor[j] = vetor[j - lacuna];
91                  trocas++;
92              }
93              vetor[j] = temp;
```

```
94          }
95      }
96      return (Resultado){trocas, (double)(clock() - inicio) /
            CLOCKS_PER_SEC};
97  }
98
99  // Auxiliares para quick e merge
100 void quickSortRec(int *vetor, int esq, int dir, long long *t) { // QUICK
        SORT //
101     if (esq < dir) {
102         int p = vetor[dir], i = esq - 1;
103         for (int j = esq; j < dir; j++) {
104             if (vetor[j] < p) {
105                 i++;
106                 int tmp = vetor[i];
107                 vetor[i] = vetor[j];
108                 vetor[j] = tmp;
109                 (*t)++;
110             }
111         }
112         int tmp = vetor[i + 1];
113         vetor[i + 1] = vetor[dir];
114         vetor[dir] = tmp;
115         (*t)++;
116         int pi = i + 1;
117         quickSortRec(vetor, esq, pi - 1, t);
118         quickSortRec(vetor, pi + 1, dir, t);
119     }
120 }
121
122 void merge(int *vetor, int esq, int meio, int dir, long long *t) {
123     int tam_esq = meio - esq + 1;
124     int tam_dir = dir - meio;
125     int *t_esq = malloc(tam_esq * sizeof(int));
126     int *t_dir = malloc(tam_dir * sizeof(int));
127
128     for (int i = 0; i < tam_esq; i++){
```

```
129         t_esq[i] = vetor[esq + i];
130     }
131     for (int i = 0; i < tam_dir; i++){
132         t_dir[i] = vetor[meio + 1 + i];
133     }
134 
135     int i = 0;
136     int j = 0;
137     int k = esq;
138     while (i < tam_esq && j < tam_dir) {
139         if (t_esq[i] <= t_dir[j]){
140             vetor[k] = t_esq[i];
141             i++;
142         } else {
143             vetor[k] = t_dir[j];
144             j++;
145         }
146         k++;
147         (*t)++;
148     }
149 
150     while (i < tam_esq) {
151         vetor[k] = t_esq[i];
152         i++;
153         k++;
154         (*t)++;
155     }
156 
157     while (j < tam_dir) {
158         vetor[k] = t_dir[j];
159         j++;
160         k++;
161         (*t)++;
162     }
163 
164     free(t_esq);
165     free(t_dir);
```

```
166 }
167
168 void mergeSortRec(int *vetor, int esq, int dir, long long *t) {
169     if (esq < dir) {
170         int meio = esq + (dir - esq) / 2;
171         mergeSortRec(vetor, esq, meio, t);
172         mergeSortRec(vetor, meio + 1, dir, t);
173         merge(vetor, esq, meio, dir, t);
174     }
175 }
176
177 // --- FUNÇÃO DE TESTE ---
178
179 void rodarTestes(char *nome_arquivo, int tam) {
180     FILE *f = fopen(nome_arquivo, "r"); //abre o arquivo que contem os
        numeros
181     if (!f) return; // sai da funcao caso nao consiga abrir o arquivo ou
         nao exista
182
183     int *orig = malloc(tam * sizeof(int));
184     int *temp = malloc(tam * sizeof(int));
185     for (int i = 0; i < tam; i++) {
186         fscanf(f, "%d", &orig[i]);
187     }
188     fclose(f);
189
190     printf("\nArquivo %s\n", nome_arquivo);
191     printf("Metodo          | Trocas        | Tempo em seg\n");
192     printf("-----------------------------------------\n");
193
194     const char *nomes[] = {"Bubble", "Insertion", "Shell", "Quick", "
        Merge"};
195     for (int i = 0; i < 5; i++) {
196         memcpy(temp, orig, tam * sizeof(int));
197         Resultado res;
198         long long t_rec = 0;
199         clock_t start;
```

```
200
201         switch(i) { // chama cada metodo de ordenacao ordenadamente
202             case 0: res = bubbleSort(temp, tam); break;
203             case 1: res = insertionSort(temp, tam); break;
204             case 2: res = shellSort(temp, tam); break;
205             case 3:  // aqui muda um pouco pq temos os metodos de
                    ordenacao recursivos (merge e quicksort)
206                 start = clock();
                    // devemos calcular o tempo aqui, pq se calculassemos
                     na funcao recursiva
207                 quickSortRec(temp, 0, tam - 1, &t_rec);
                                                    // ele iria medir o
                    tempo de cada iteracao da chamada recursiva
208                 res = (Resultado){t_rec, (double)(clock() - start) /
                    CLOCKS_PER_SEC};
209                 break;
210             case 4:
211                 start = clock();
212                 mergeSortRec(temp, 0, tam - 1, &t_rec);
213                 res = (Resultado){t_rec, (double)(clock() - start) /
                    CLOCKS_PER_SEC};
214                 break;
215         }
216         printf("%-15s | %-12lld | %-10.4f\n", nomes[i], res.trocas, res.
            tempo);
217     }
218     free(orig); free(temp);
219 }
220
221 int main() {
222     gerarArquivos();
223
224     // chama todos os testes
225     int tamanhos[] = {1000, 10000, 100000};
226     for (int t = 0; t < 3; t++) {
227         for (int f = 1; f <= 5; f++) {
```

```
228             char nome[30];
229             sprintf(nome, "%d-arq%d.txt", tamanhos[t], f);
230             rodarTestes(nome, tamanhos[t]);
231         }
232     }
233 
234     return 0;
235 }
```